

MiniLLM: Knowledge Distillation of LLMs

Dhruv Kumar

BITS Pilani

How language models are normally trained

An LLM is trained to do one thing, over and over: given the tokens seen so far, predict a probability distribution over the **entire vocabulary** for what comes next.

ONE TRAINING STEP

Take a real sentence, e.g. "the cat sat". At the position after "the cat", the model outputs a probability for *every* word in the vocabulary — "sat", "ran", "dog", all of them. Training nudges the weights so this distribution puts more probability on the word that *actually* comes next in the real sentence — repeated at every position, of every sentence, in the whole training set.

The question this whole deck is really about: what if, instead of "the word that actually comes next," the target the model is pushed toward is a **teacher model's opinion** of what should come next?

The loss: cross-entropy against one-hot truth

The "ground truth" at each position is a **one-hot vector** y — probability **1** on the actual next token, **0** on every other word in the vocabulary. The loss is cross-entropy between the model's predicted distribution and this one-hot target:

$$\mathcal{L} = - \sum_{v \in V} y_v \log q_{\theta}(v \mid y_{<t}, x) = - \log q_{\theta}(y_t \mid y_{<t}, x)$$

V — the vocabulary; $y_v = 1$ for the true next token, **0** otherwise, so every term except the true token's vanishes; $q_{\theta}(y_t \mid y_{<t}, x)$ — the probability the model actually assigned to the real next token. Minimizing this is exactly maximum-likelihood training: push the probability of the real data as high as possible.

A worked example: one token's loss

Vocabulary {the, cat, sat, dog, ran}; context "the cat ____", true next word "sat". Suppose the model's predicted distribution is $q_\theta = [0.05, 0.10, 0.60, 0.05, 0.20]$:

WORD	THE	CAT	SAT (TRUE)	DOG	RAN
y (one-hot)	0	0	1	0	0
q_θ	0.05	0.10	0.60	0.05	0.20
$-y \log q_\theta$	0	0	0.5108	0	0

$\mathcal{L} = -\log(0.60) = 0.5108$ – only the true token's probability mattered; the model's other **0.40** of probability mass (spread over "cat", "dog", "ran") never enters the loss at all, no matter how it's distributed among the wrong answers.

Standard knowledge distillation: soft targets

Replace the one-hot target with the **teacher's own predicted distribution** — same cross-entropy formula, different target:

$$\mathcal{L}_{\text{KD}} = - \sum_{v \in V} p(v \mid y_{<t}, x) \log q_{\theta}(v \mid y_{<t}, x)$$

$p(v \mid \dots)$ — the teacher's probability for word v , now used as the target instead of a one-hot vector. Often softened with a **temperature** T first: $q_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$,

z the model's raw logits — $T = 1$ is ordinary softmax; $T > 1$ spreads probability mass out, revealing the teacher's relative confidence across the *wrong* answers too — "dark knowledge" a one-hot label can never carry.

A worked example: word-level KD loss

Same context, same student q_θ . Now the teacher's own distribution is the target — say the teacher also rates "ran" fairly plausible ("the cat ran" makes sense too):

WORD	THE	CAT	SAT	DOG	RAN
p (teacher)	0.02	0.05	0.70	0.03	0.20
q_θ (student)	0.05	0.10	0.60	0.05	0.20
$-p \log q_\theta$	0.0599	0.1151	0.3576	0.0899	0.3219

$\mathcal{L}_{\text{KD}} = 0.9444$ — every word contributes now, including "ran" (**0.20** of the teacher's belief lands there). The student is pushed toward the teacher's *whole shape*, not just the one correct answer from before.

This is exactly forward KLD

Split the cross-entropy into two pieces, using an identity true for *any* two distributions — not something special to KD:

$$\mathcal{L}_{\text{KD}} = \underbrace{-\sum_v p_v \log p_v}_{H(\text{teacher}), \text{ fixed}} + \underbrace{\sum_v p_v \log \frac{p_v}{q_{\theta,v}}}_{\text{KL}[p \parallel q_{\theta}]}$$

$H(\text{teacher})$ — the teacher's own **Shannon entropy**, how spread-out p itself is, computed with no student involved at all. Cross-entropy measures the average "surprise" of an outcome drawn from p , scored using q_{θ} 's probabilities — even a *perfect* student ($q_{\theta} = p$) leaves $H(\text{teacher})$ of irreducible surprise behind, since p itself isn't a certainty.

VERIFIED: THE LOSS FLOOR IS $H(\text{TEACHER})$, NOT 0

Student $\neq$ teacher: $H = 0.9048$, $\text{KL} = 0.0396$, $\text{sum} = 0.9444 = \mathcal{L}_{\text{KD}}$. Student set to match the teacher *exactly*: $\text{KL} = 0$, so $\mathcal{L}_{\text{KD}} = H(\text{teacher}) = 0.9048$ — not 0. Since $H(\text{teacher})$ never involves θ , it contributes zero gradient — **minimizing the KD loss and minimizing forward KLD are the same optimization problem**, they just report different numbers. This was all at one token — the next slide zooms out to a whole response.

Knowledge distillation: teacher to student

Zooming out from one token to the whole task: knowledge distillation (KD) trains a small **student** model using supervision from a large, already-trained **teacher** model, instead of (or in addition to) raw labeled data — the point is to shrink the compute needed at inference time.

Black-box KD

Only the teacher's generated *text* is available (e.g. an API like ChatGPT). The student trains on teacher outputs directly.

White-box KD

The teacher's full output *distribution* is available — exactly the setting from the previous slides. The student can match probabilities, not just copy text.

This paper studies **white-box KD for open-source LLMs** — small models trained to match a large model's actual output distribution.

The standard objective: forward KLD

Given a prompt x , the teacher's output distribution $p(y|x)$ over responses y , and a student $q_\theta(y|x)$, standard KD approximately minimizes the **forward** Kullback-Leibler divergence:

$$\text{KL}[p \parallel q_\theta] = \mathbb{E}_{x \sim p_x, y \sim p} \log \frac{p(y|x)}{q_\theta(y|x)}$$

x — a prompt, sampled from the prompt distribution p_x ; y — a full response; $p(y|x)$ — the teacher's probability of response y ; $q_\theta(y|x)$ — the student's probability of the same response, parameterized by weights θ . This is the same objective as ordinary maximum-likelihood training, just against the teacher's distribution instead of ground-truth labels.

From sequences to tokens: the chain rule

$p(y|x)$ is a probability over an entire response — but the teacher only ever hands you *per-token* distributions, the same ones from every earlier slide. The bridge is the chain rule: the probability of a whole sequence is the product of each token's conditional probability, given everything before it:

$$p(y|x) = \prod_{t=1}^T p(y_t|y_{<t}, x) \implies \log \frac{p(y|x)}{q_\theta(y|x)} = \sum_{t=1}^T \log \frac{p(y_t|y_{<t}, x)}{q_\theta(y_t|y_{<t}, x)}$$

Extend the running example by one token, "the cat sat down":

STEP	CONTEXT → TRUE TOKEN	P	Q_θ	$\text{LOG}(P/Q_\theta)$
1	"the cat" → "sat"	0.70	0.60	0.1542
2	"the cat sat" → "down"	0.70	0.50	0.3365

$p(y|x) = 0.70 \times 0.70 = 0.49$, $q_\theta(y|x) = 0.60 \times 0.50 = 0.30$, $\log(0.49/0.30) = 0.4906$ — exactly **0.1542 + 0.3365**. The sequence-level ratio in the forward-KLD formula **is** a sum of per-token log-ratios — no new quantity to compute, just the per-token terms already familiar, added up along the response.

Word-level KD vs. SeqKD: two approximations

Forward KLD wants two things at once: y sampled from the **teacher's own generation**, and the student matching the teacher's **full distribution** at every step. Neither practical method gets both — they trade off in opposite directions:

	CONTEXT $Y_{<T}$ COMES FROM	PER-TOKEN TARGET
Word-level KD	real, ground-truth data	teacher's full soft distribution (exact)
SeqKD	the teacher itself (exact)	one-hot on the teacher's own sampled token

Word-level KD keeps the soft, per-token target exact, but evaluates it on real sequences instead of teacher-generated ones — the paper's "approximately," since real data only matches teacher-generated contexts once the teacher is strong. **SeqKD** keeps the sampling source exact — a real teacher-generated sequence — but then discards the teacher's actual probabilities and trains on that one sample as ordinary one-hot ground truth, losing the dark knowledge every other slide has been carrying.

Why not sample from the teacher and keep the soft target?

Nothing rules this out mathematically — sample y from the teacher, then match the teacher's full distribution at every step of that same y . That is exactly the un-approximated forward-KLD objective from three slides ago. Each practical method drops it for its own documented reason:

SEQKD: THE EXACT TARGET IS AN INTRACTABLE SUM

The true sequence-level target sums over **every** possible response, weighted by how likely the teacher is to produce it: $-\sum_{y \in \mathcal{Y}} p(y|x) \log q_\theta(y|x)$ — exponentially many terms. Kim & Rush's original SeqKD approximates $p(y|x)$ by a single point, the teacher's most likely response $\hat{y}$ (found via beam search). Collapsing the teacher's distribution down to one response is exactly what removes the soft target — the loss becomes plain $-\log q_\theta(\hat{y}|x)$, with nothing soft left to keep.

WORD-LEVEL KD: REAL DATA IS ALREADY THERE

Scoring the teacher's per-token distribution on an *already-written* sequence takes one parallel forward pass. Generating a *new* response from the teacher instead takes T sequential decoding steps per example, repeated across the whole training set — so word-level KD keeps using the real dataset it already has.

SELF-STUDY: ON-POLICY DISTILLATION

Later work (Agarwal et al., 2024) shows this combination — a generated sequence *plus* the teacher's per-token soft distribution along it — is a real option, but works better when the sequence comes from the **student**, not the teacher: that is what the student needs to get right at inference time, the same reason MiniLLM samples y from q_θ on the next slide.

Why forward KLD hurts open-ended generation

Forward KLD forces q_θ to place probability **everywhere** p does — if $p(y|x) > 0$ but $q_\theta(y|x) \rightarrow 0$ anywhere, $\log(p/q_\theta) \rightarrow \infty$ and the loss explodes. For a small number of classes (text classification), this is harmless: both distributions have few modes anyway.

TEXT GENERATION HAS TOO MANY MODES FOR A SMALL STUDENT

An LLM's response space is enormous and multi-modal — many different phrasings are all individually plausible. A small student lacks the capacity to represent every mode p does, so forced coverage makes it spread probability onto **void regions** of p — places the teacher finds implausible — producing low-quality, hallucinated generations at free-run time.

A worked example: forward KLD spills into the void

Context "the cat ___", a 4-token vocabulary. The teacher genuinely likes two different continuations — "sat" and "ran" — and almost never says "purred" or "flew":

TOKEN	SAT	PURRED	FLEW	RAN
teacher p	0.49	0.01	0.01	0.49

Suppose the student is capacity-limited to a single "bump" of confidence across these four tokens — it can commit to one coherent continuation shape, not two unrelated ones at once. Fitting that one bump by forward vs. reverse KLD gives very different students:

FORWARD KLD: $\text{ARG MIN}_Q \text{KL}[P \parallel Q]$

$q = [0.03, \mathbf{0.47}, \mathbf{0.47}, 0.03]$ — the bump lands **between** the two real modes, the only place a single bump gets partial credit for both. 94% of the student's probability lands on "purred"/"flew".

REVERSE KLD: $\text{ARG MIN}_Q \text{KL}[Q \parallel P]$

$q = [\mathbf{0.80}, 0.20, 0.00, 0.00]$ — the bump commits to one real mode, "sat", and stays close to it. Only 20% spills onto the neighboring void token, none onto the far one.

Same teacher, same student family — minimizing forward KLD produces a student that mostly says implausible things; minimizing reverse KLD produces one that says a real thing, just not always the same real thing.

MiniLLM's objective: reverse KLD

MiniLLM instead minimizes the **reverse** KLD — the same two distributions, with the direction flipped:

$$\theta = \arg \min_{\theta} \text{KL}[q_{\theta} \parallel p] = \arg \min_{\theta} \left[- \mathbb{E}_{x \sim p_x, y \sim q_{\theta}} \log \frac{p(y|x)}{q_{\theta}(y|x)} \right]$$

Same p , q_{θ} , x as before — but now y is sampled from the **student's own** distribution q_{θ} , not the teacher's. This one change — whose distribution y comes from — is what turns the training signal from "cover everything the teacher might say" into "get good at what you already tend to say."

Forward vs. reverse KLD: a numeric example

KL divergence is not symmetric — $\text{KL}[p||q] \neq \text{KL}[q||p]$ in general. Take $p = [0.70, 0.25, 0.05]$ and $q = [0.40, 0.40, 0.20]$ over 3 outcomes:

<i>Y</i>	<i>P</i>	<i>Q</i>	<i>P</i> LOG (<i>P</i> / <i>Q</i>)	<i>Q</i> LOG (<i>Q</i> / <i>P</i>)
1	0.70	0.40	0.3917	−0.2238
2	0.25	0.40	−0.1175	0.1880
3	0.05	0.20	−0.0693	0.2773
sum			KL[p q] = 0.2049	KL[q p] = 0.2414

Same two distributions, two genuinely different numbers — forward and reverse KLD are not interchangeable objectives; minimizing one does not minimize the other.

Why: which mistake each direction punishes

Look at what each term does when p and q disagree in different ways:

Q SPENDS PROBABILITY WHERE P IS TINY ($Q = 0.20$, $P = 0.01$)

reverse term $q \log(q/p) = 0.20 \log(20) = \mathbf{0.599}$ — big penalty.

forward term $p \log(p/q) = 0.01 \log(0.05) = -0.030$ — negligible.

Q IGNORES A REGION P LIKES ($Q = 0.001$, $P = 0.30$)

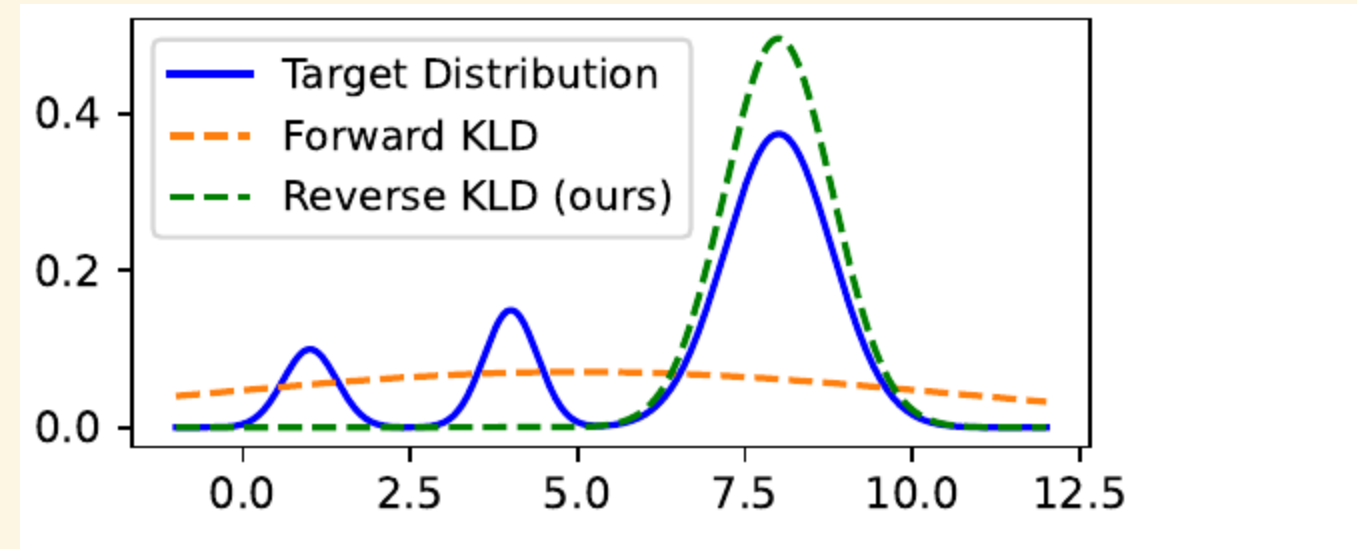
forward term $p \log(p/q) = 0.30 \log(300) = \mathbf{1.711}$ — big penalty.

reverse term $q \log(q/p) = 0.001 \log(0.0033) = -0.006$ — negligible.

Forward KLD is severely punished for under-covering any mode p has — mode-*covering*. **Reverse KLD** is severely punished for wandering into regions p dislikes, but barely cares about ignoring modes — mode-*seeking*. This asymmetry is the entire reason the two objectives train such differently-behaved students.

Seeing it directly: fitting a single Gaussian

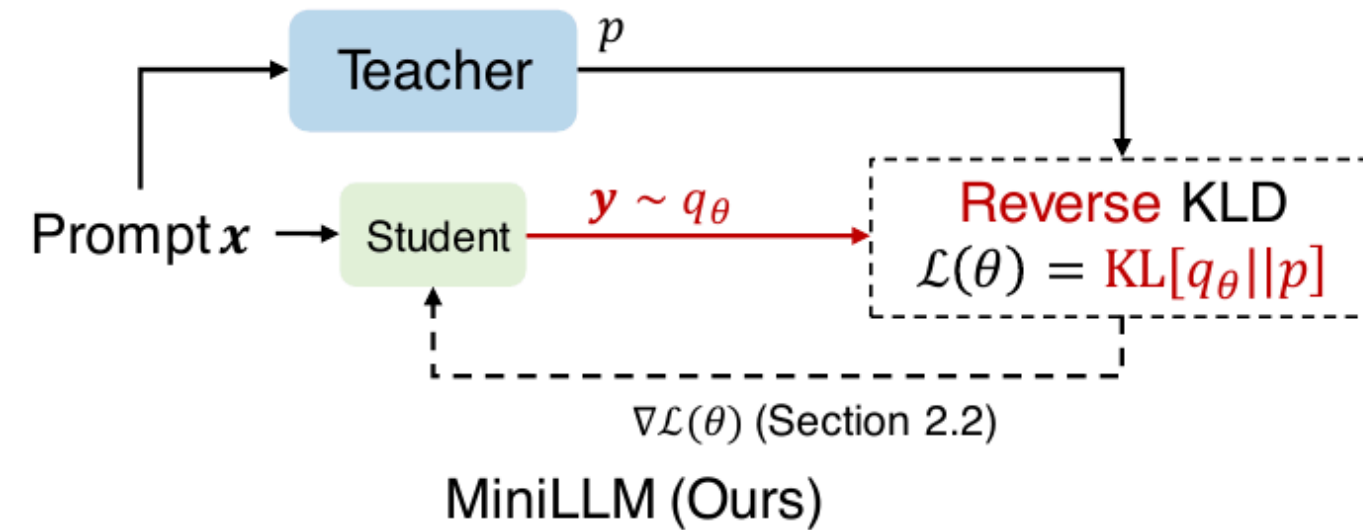
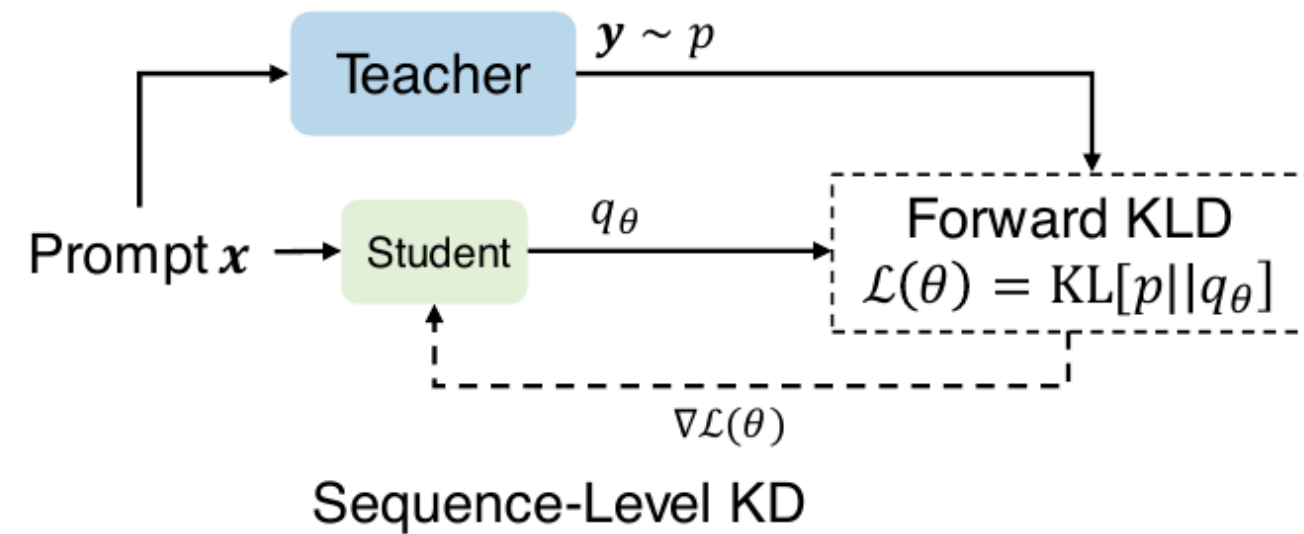
Fit one Gaussian q to a multi-modal target p (a Gaussian mixture) by minimizing each KLD direction:



Forward KLD (orange) spreads itself thin trying to cover every mode at once, ending up low and wide — reasonably present everywhere, confidently right nowhere.

Reverse KLD (green) picks the single largest mode and matches it closely, ignoring the smaller ones entirely — exactly the mode-seeking behavior from the previous slide, now visible on an actual distribution.

What this means for training



Sequence-level KD samples y from the **teacher** and trains the student to match those exact samples — forward KLD, forcing memorization of everything the teacher might generate. MiniLLM samples y from the **student itself** and asks the teacher to grade it — reverse KLD, improving the text the student already tends to produce rather than forcing it to reproduce the teacher's.

Why this needs policy gradient

$\text{KL}[q_\theta || p]$ is an expectation *over* $y \sim q_\theta$ — the distribution being optimized is also the one doing the sampling. Ordinary backprop can't differentiate through a sampling step directly. This is exactly the situation reinforcement learning solves: treat the student as a **policy** choosing tokens, use the **Policy Gradient Theorem** to get a usable gradient anyway.

THE RL FRAMING

State = the prompt plus tokens generated so far, $(x, y_{<t})$. Action = the next token y_t , chosen from the vocabulary. Reward = how much more the teacher likes each generated token than the student already does.

The gradient, qualitatively

The Policy Gradient Theorem gives an update rule with the shape every policy-gradient method shares: push up the probability of an action in proportion to a **reward** signal, push it down otherwise. Here, at each generated token y_t , the reward is R_t — how much more the teacher likes the *rest* of this response, from this point on, than the student itself currently expects.

THE PRECISE MECHANICS COME LATER

This deck hasn't covered RL for LLMs / RLHF yet, so the full derivation of R_t and the gradient formula is deliberately left qualitative here — worth revisiting in detail once that foundation is in place. The idea to hold onto for now: reward = teacher-relative quality of what follows, and the gradient just does ordinary policy-gradient credit assignment with that reward.

Three problems, three fixes

Plain policy gradient on this reward is unstable in practice. The paper identifies three specific failure modes and a fix for each — the mechanics are RL-specific and worth returning to once RL for LLMs is covered properly; the ideas themselves are simple:

High variance

Early-token errors compound across a whole sampled sentence. Fix: compute the immediate next-step quality exactly (summed over the vocabulary) instead of estimating it from one noisy rollout.

Reward hacking

Sampling purely from the student can find degenerate text (repeated phrases) that the teacher scores well. Fix: sample from a mix of teacher and student, so training rarely wanders into those degenerate regions in the first place.

Length bias

Summing reward over more remaining tokens mechanically favors short or empty responses. Fix: normalize the reward by how many tokens remain, so it reflects average quality, not raw length.

Training MiniLLM: two phases

1. **Phase 1 — SFT initialization.** Fine-tune the student on the labeled instruction-response dataset $\mathcal{D}$, standard supervised training. This gives policy gradient a reasonable starting point instead of a random one.
2. **Phase 2 — MiniLLM refinement.** Continue training with the combined gradient from the three fixes above (single-step + teacher-mixed + length-normalized), alongside a language-modeling loss on a separate pretraining corpus $\mathcal{D}_{\text{PT}}$ to preserve the student's general capabilities.

The overall pipeline mirrors RLHF: an SFT phase followed by a policy-optimization phase — except the reward here comes directly from the teacher's own output distribution, not a separately-trained reward model.

Experimental setup

Instruction-following as the task — models learn to produce a response given an instruction, evaluated with Rouge-L against ground truth, human judgment, and GPT-4 feedback.

Teacher / student pairs

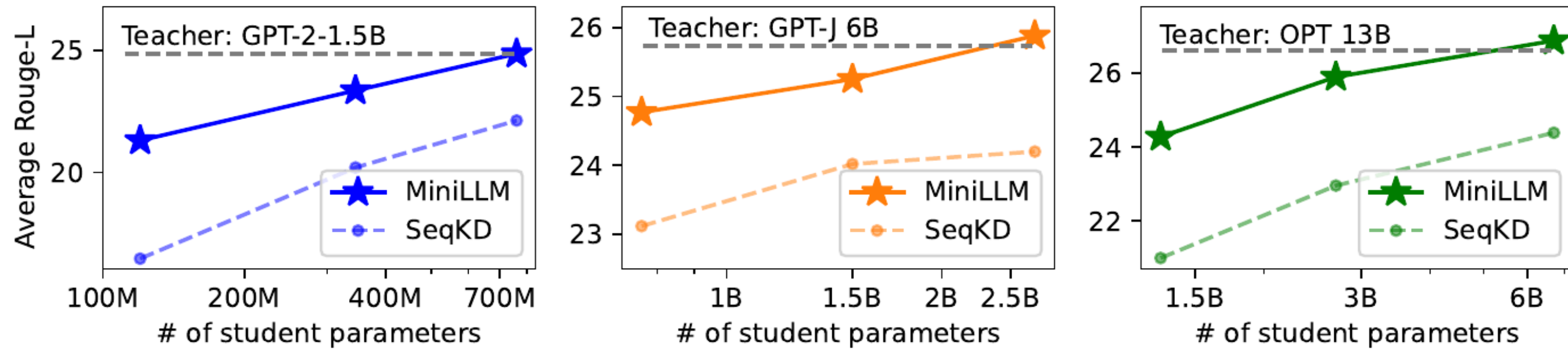
GPT-2-1.5B → GPT-2 125M/340M/760M
GPT-J 6B / OPT 13B → OPT 1.3B/2.7B/6.7B
LLaMA 13B → LLaMA 7B

Data & evaluation

Trained on databricks-dolly-15K (~12.5K examples). Evaluated on 5 instruction-following sets: Dolly, SelfInst, Vicuna, S-NI, UnNI.

Baselines: **SFT w/o KD** (no distillation at all), **KD** (word-level, forward KLD), **SeqKD** (sequence-level, trains directly on teacher-generated text).

Does it work? The headline result



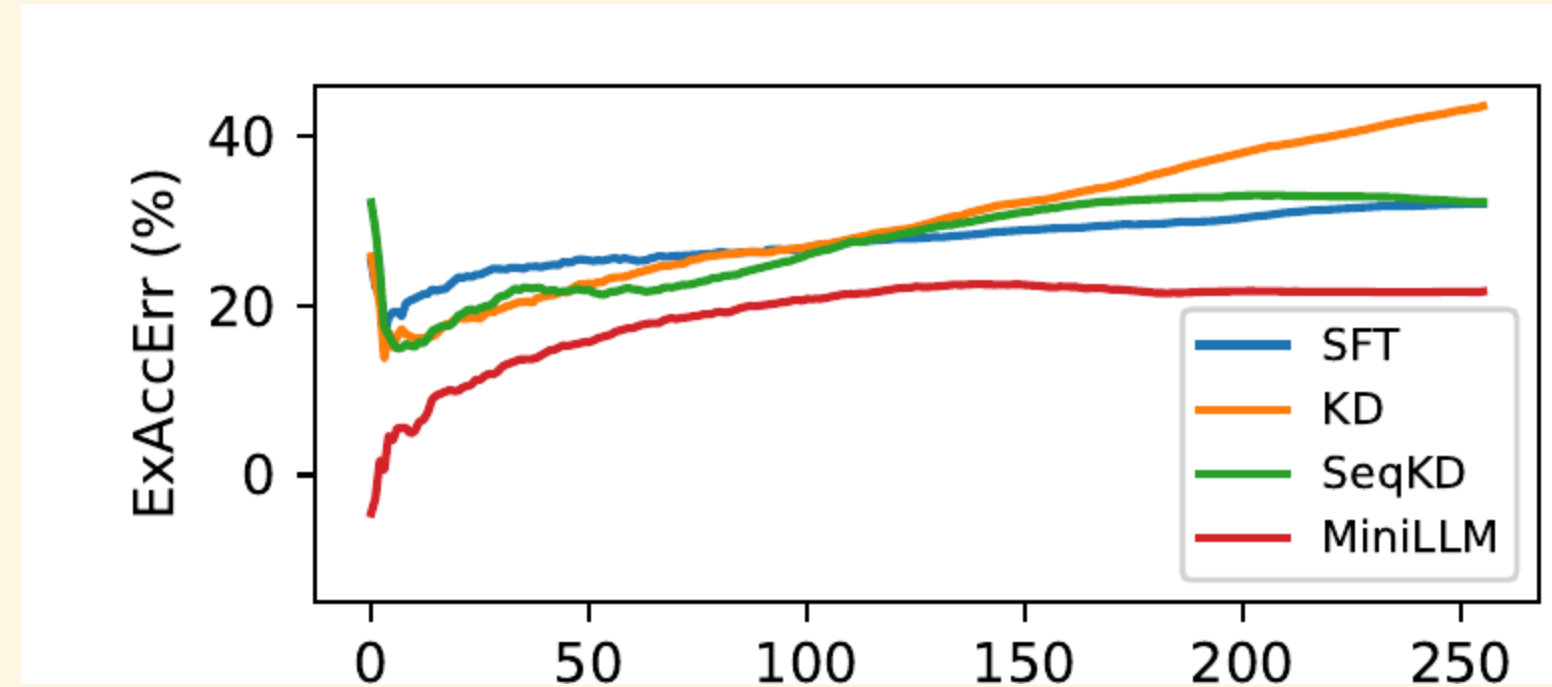
Average Rouge-L vs. student size, three independent teacher/student families. MiniLLM (solid) beats SeqKD (dashed) at every size tested, and the gap holds as student capacity scales from 100M to 6.7B — approaching, and at the largest sizes nearly matching, the teacher's own score (dashed grey line).

The full results table

Model	#Params	Method	Dolly	SelfInst	Vicuna	S-NI	UnNI
GPT-2	1.5B	Teacher	27.6	14.3	16.3	27.6	31.8
	120M	SFT w/o KD	23.3	10.0	14.7	16.3	18.5
		KD	22.8	10.8	13.4	19.7	22.0
		SeqKD	22.7	10.1	14.3	16.4	18.8
		MINILLM	24.6	13.2	16.9*	25.3	26.6
	340M	SFT w/o KD	25.5	13.0	16.0	25.1	28.1
		KD	25.0	12.0	15.4	23.7	24.6
		SeqKD	25.3	12.6	16.9*	22.9	23.3
		MINILLM	25.4	15.6*	17.7*	27.4	30.8
	760M	SFT w/o KD	25.4	12.4	16.1	21.5	24.0
		KD	25.9	13.4	16.9*	25.3	28.0
		SeqKD	25.6	14.0	15.9	26.1	29.1
		MINILLM	26.4	15.9*	18.3*	29.3*	34.5*
OPT	13B	Teacher	29.2	18.4	17.8	30.4	36.1
	1.3B	SFT w/o KD	26.0	11.4	15.6	23.1	28.4
		KD	25.4	12.2	14.9	21.9	27.0
		SeqKD	26.1	12.7	16.6	21.4	28.2
		MINILLM	26.7	14.8	17.9*	28.6	33.4
	2.7B	SFT w/o KD	27.1	13.9	16.6	24.9	32.3
		KD	25.9	13.8	16.7	26.3	30.2
		SeqKD	27.5	13.3	16.5	25.3	32.3
		MINILLM	27.4	17.2	19.1*	30.7*	35.1
	6.7B	SFT w/o KD	27.6	16.4	17.8	30.3	28.6
		KD	28.3	17.0	17.5	30.7*	26.7
		SeqKD	28.5	17.0	17.9*	30.4	28.2
		MINILLM	29.0	17.5	18.7*	32.5*	36.7*
LLaMA	13B	Teacher	29.7	23.4	19.4	35.8	38.5
	7B	SFT w/o KD	26.3	20.8	17.5	32.4	35.8
		KD	27.4	20.2	18.4	33.7	37.9
		SeqKD	27.5	20.8	18.1	33.7	37.6
		MINILLM	29.0	23.2	20.7*	35.5	40.2*

MiniLLM is the best or tied-best student almost everywhere, across every family and every size. On several datasets (Vicuna, S-NI, UnNI) it even exceeds the teacher's own score — attributed to **exposure bias**: the teacher was fine-tuned with plain teacher-forcing, which itself has a training/inference mismatch that policy-gradient training doesn't share (next slide).

Lower exposure bias



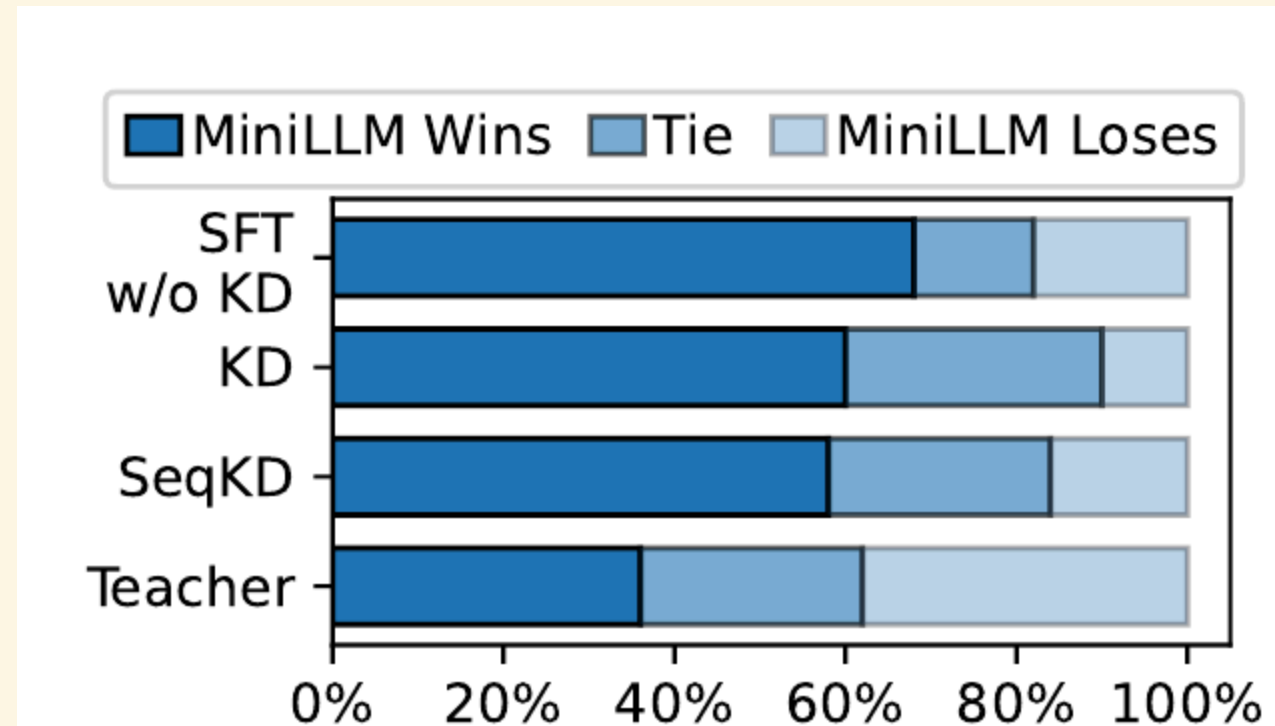
Teacher-forced training only ever sees ground-truth prefixes, never its own mistakes — at free-run inference, small errors compound as generation gets longer (exposure bias). MiniLLM trains on the student's *own* generated rollouts (reverse KLD samples $y \sim q_\theta$), so it already sees, and corrects for, its own mistakes during training — the excess error (ExAccErr) stops growing past ~150 tokens, while the baselines keep drifting.

Better calibration

	SST2		BoolQ	
	ECE	Acc.	ECE	Acc.
Teacher	0.025	93.0	0.356	74.5
KD	0.191	84.7	0.682	63.5
SeqKD	0.243	66.5	0.681	62.8
MINILLM	0.099	89.7	0.502	67.8

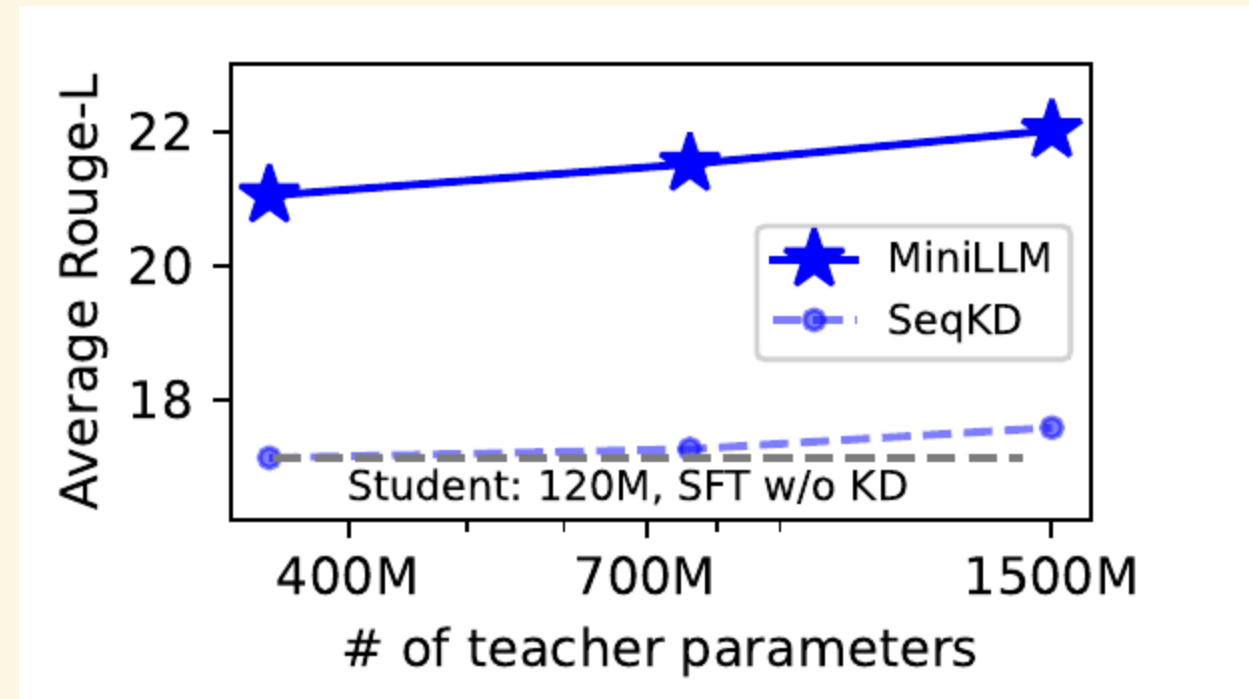
Expected Calibration Error (ECE) — how well a model's confidence matches its actual accuracy — on two zero-shot classification tasks. Forward-KLD baselines (KD, SeqKD) are dramatically worse calibrated than the teacher; MiniLLM narrows that gap substantially, consistent with reverse KLD avoiding the void-region overconfidence forward KLD introduces.

Human preference



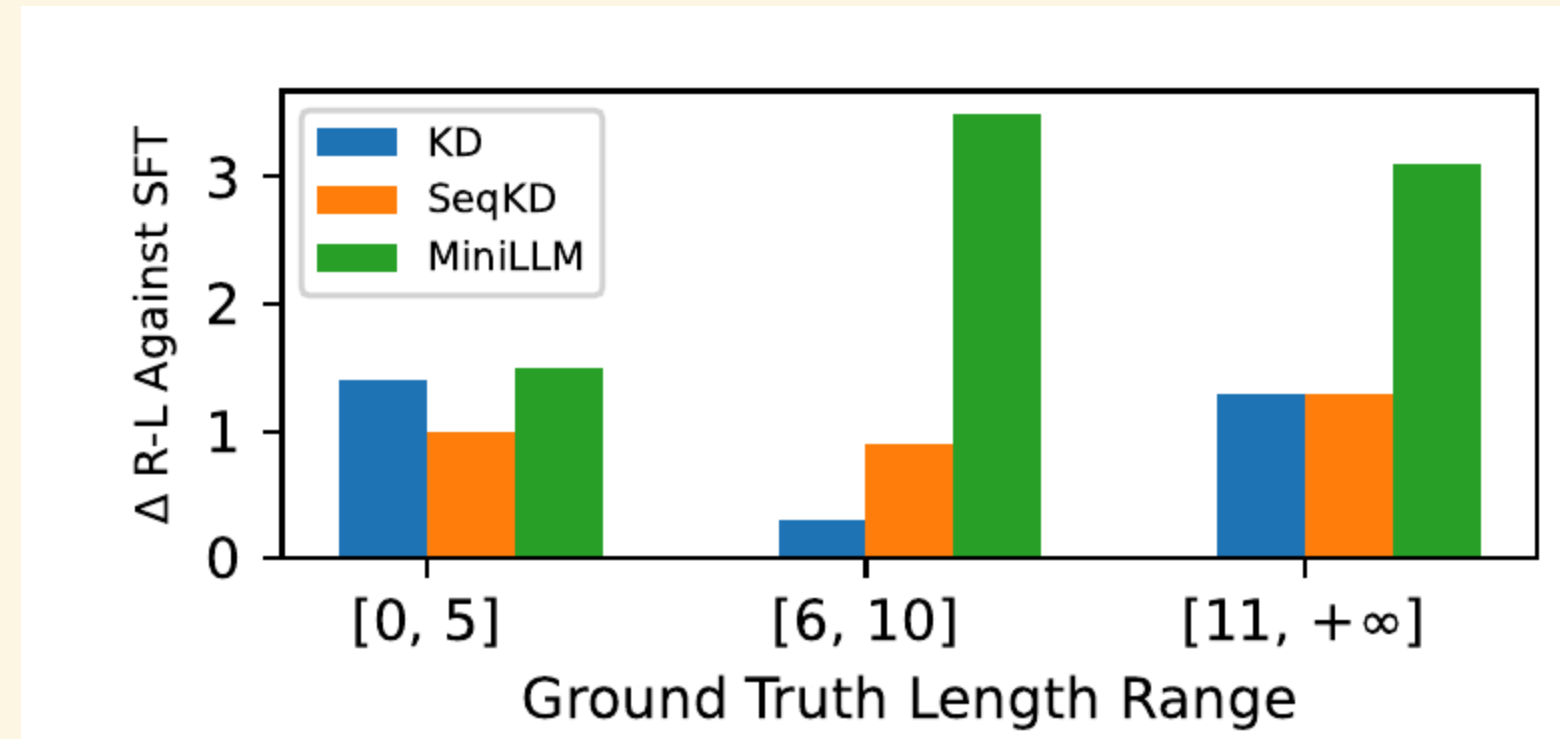
Head-to-head human comparisons, LLaMA-7B student vs. LLaMA-13B teacher. MiniLLM wins outright against every baseline more often than it loses, and even against the **teacher itself** it's competitive — wins or ties in roughly two-thirds of comparisons.

Bigger teachers make better students



Fix the student (GPT-2-125M) and grow only the teacher. Prior work has found this doesn't always help — a bigger teacher can even hurt naive distillation. MiniLLM instead improves *monotonically* with teacher size, while SeqKD stays flat — reverse KLD keeps extracting more signal from a stronger teacher instead of just inheriting its blind spots.

The advantage grows with response length



For short expected responses (≤ 5 tokens) the output space is small enough that any method covers it — forward and reverse KLD perform similarly. For longer responses, the teacher's true distribution has far more modes than a small student can represent, and that's exactly where MiniLLM's margin over standard KD widens the most.

Does mode-seeking hurt diversity?

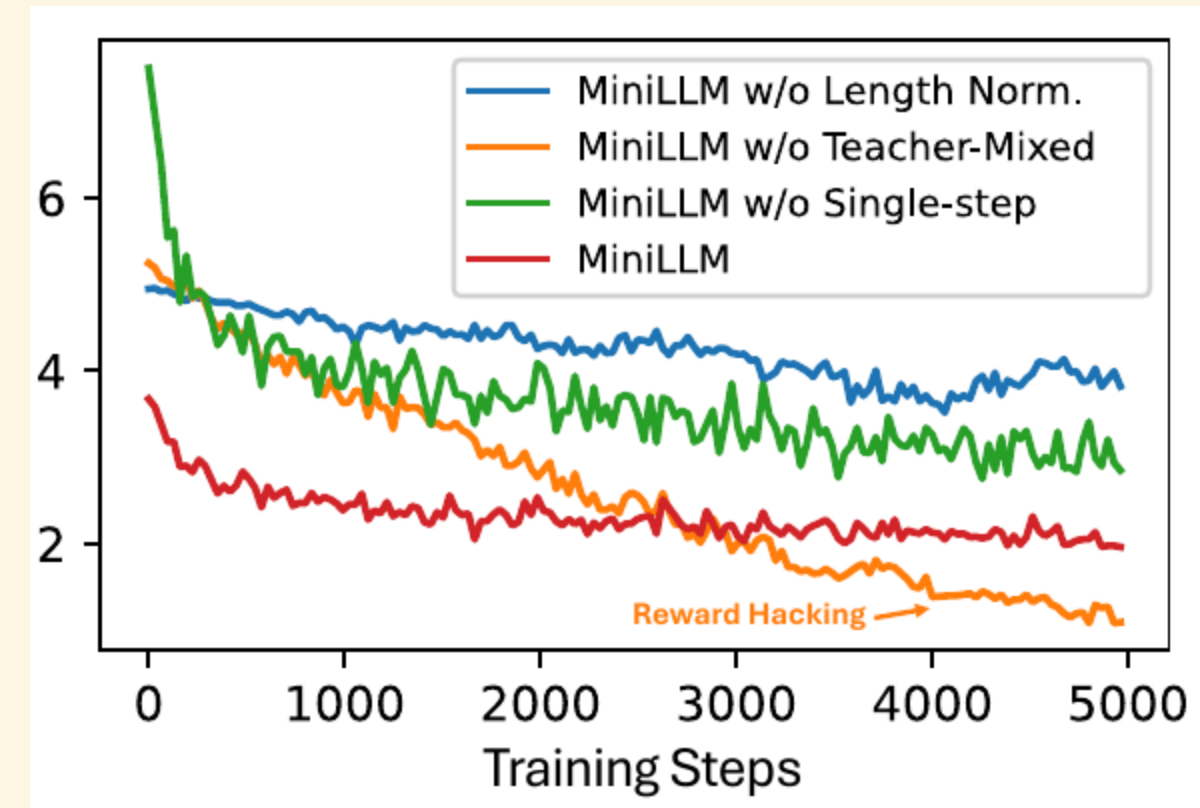
Reverse KLD is mode-seeking by design — a natural worry is that it makes generations repetitive. Measured directly with distinct-4-grams (Dist-4, higher = more diverse vocabulary) and language-modeling loss on held-out text:

	Dolly		SelfInst	
	Dist-4	Loss	Dist-4	Loss
Teacher	99.3	3.55	99.1	4.44
KD	99.4	3.93	98.8	5.36
SeqKD	99.3	3.91	98.8	5.22
MINILLM	99.0	3.95	98.6	5.33

MiniLLM's Dist-4 and loss stay close to the teacher's and the other baselines' — mode-seeking within one response does not collapse diversity *across* different prompts or different sampling seeds.

Ablations: does every fix earn its place?

	Valid. R-L	Dolly R-L
MINILLM	27.4	24.6
w/o Length Norm.	17.4	14.7
w/o Teacher-Mixed	22.3	20.4
w/o Single-Step	27.0	23.7



Removing **teacher-mixed sampling** hurts the most (27.4→17.4 validation Rouge-L) — without it, reverse KLD still goes down during training (right plot, orange), but the model is reward-hacking: learning short, repeated, degenerate strings the teacher happens to score well, not real quality. Single-step decomposition mainly reduces variance (smoother curves, higher final scores) rather than changing the endpoint qualitatively.

What the outputs actually look like

Case #1	
Instruction	During writing, we added an asterisk for the word that did not come to mind. You will need to provide several examples to demonstrate all the words that can be used in the sentence instead of the asterisk.
Input	we * to know
SFT w/o KD	We can use “know” in the sentence instead of “we * to know”
KD	we went to know\n we will go to know\n we can go to know\n we may go to know\n we should go to know\n we must go to know
SeqKD	we do to know\n we can do to know\n we should do to know\n we would do to know\n we ought to do to know
MINILLM	We can use the following words in place of the asterisk:\n - want\n - have\n - do\n - used\n - ought\n - desire\n - need\n - must
MINILLM w/o teacher-mixed	We to know

MiniLLM produces the most complete, correctly-formatted answer — a full bulleted list of valid substitutions. Without teacher-mixed sampling, the same architecture degenerates into a near-empty response ("We to know") — a direct, visible instance of the reward hacking the ablation on the previous slide measured numerically.

Reading list

- [Gu, Dong, Wei & Huang, *MiniLLM: Knowledge Distillation of Large Language Models*](#) — ICLR 2024, the source for every figure and result in this deck.
- [Hinton, Vinyals & Dean, *Distilling the Knowledge in a Neural Network*](#) — the original knowledge distillation paper.
- [Huszár, *How \(not\) to train your generative model*](#) — forward vs. reverse KL and mode-covering/seeking behavior in generative modeling.
- [Ouyang et al., *Training Language Models to Follow Instructions with Human Feedback*](#) — the RLHF pipeline MiniLLM's two-phase training mirrors.
- [Kim & Rush, *Sequence-Level Knowledge Distillation*](#) — the original SeqKD paper.
- [Agarwal et al., *On-Policy Distillation of Language Models*](#) — trains the student on its own generated sequences, graded by the teacher.
- [Official MiniLLM code, data, and model checkpoints.](#)